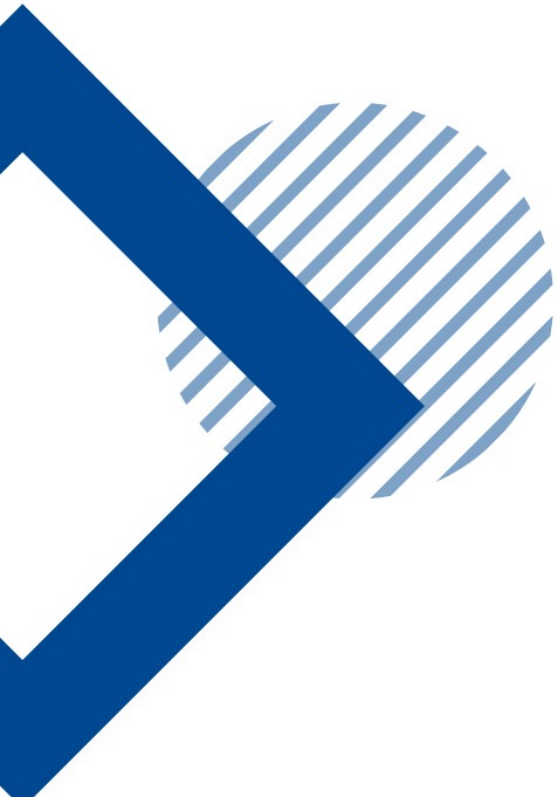
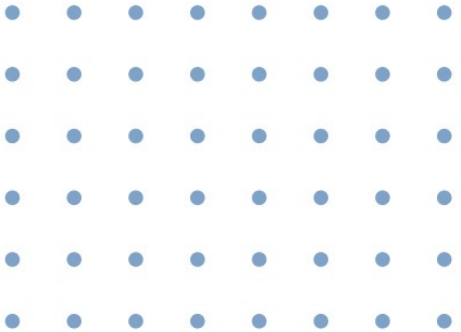
Web-сервисы

Интернет программирование







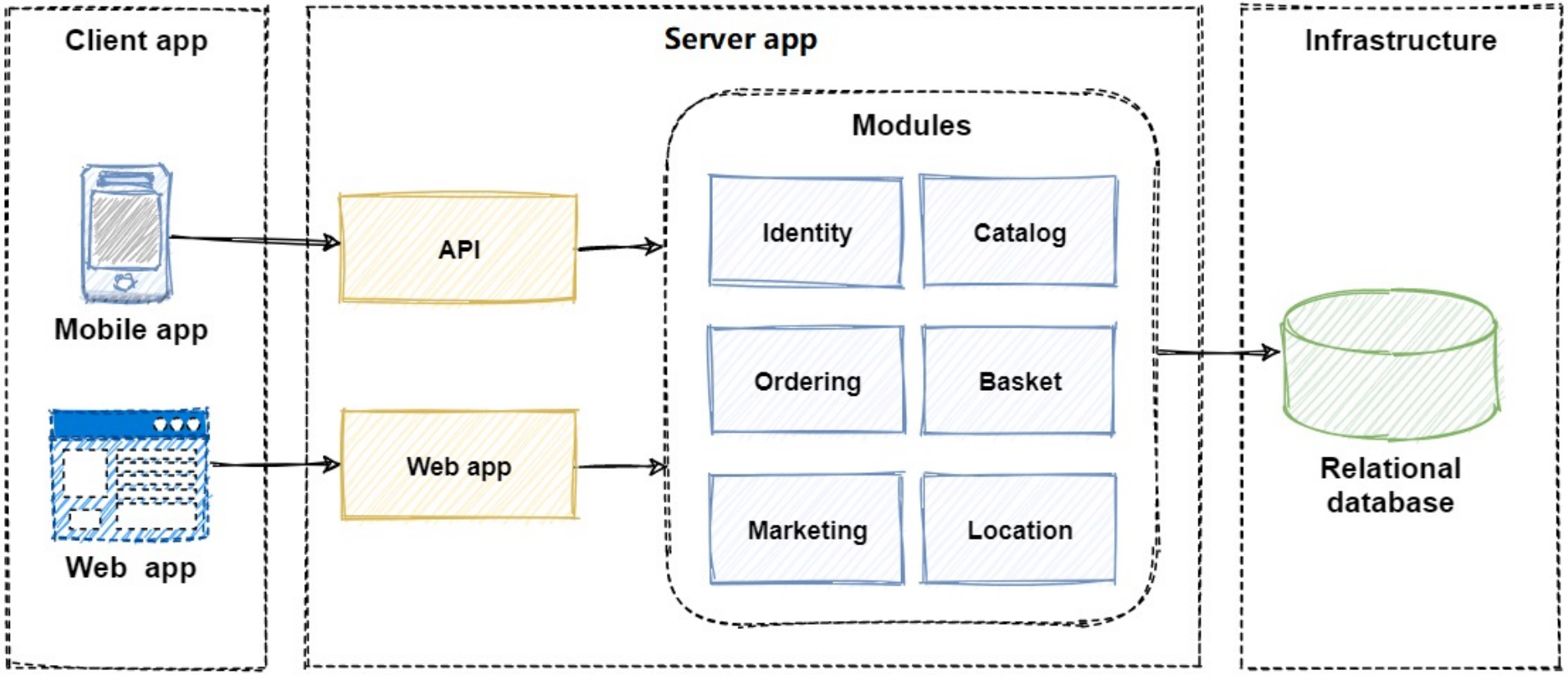
2022

Распределенное приложение

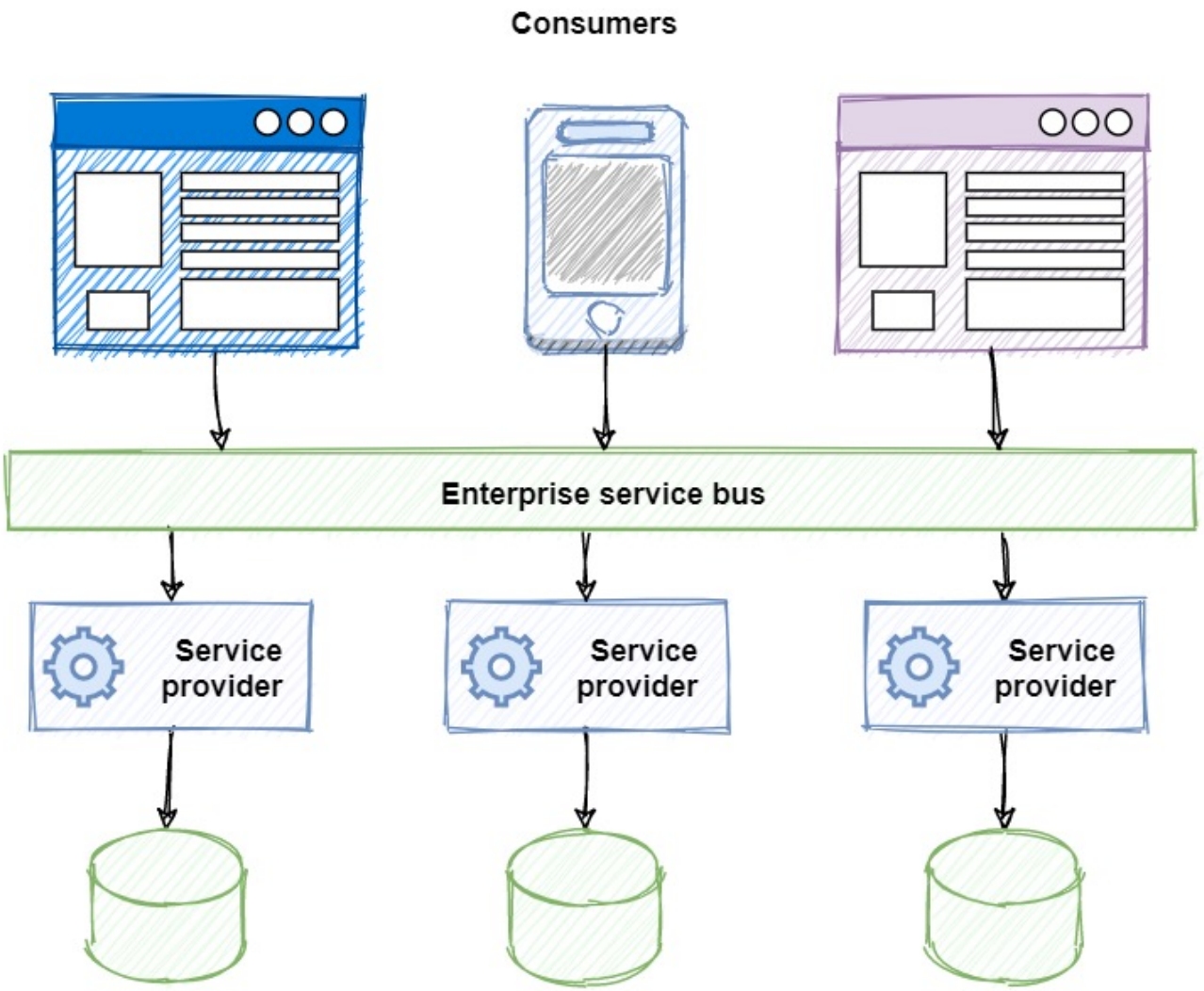
Распределенное приложение – приложение, состоящее из отдельных процессов (узлов), взаимодействующих между собой путем обмена сообщениями через каналы связи (сеть, сокеты, межпроцессорное взаимодействие).

Варианты архитектуры распределенного приложения:
- клиент-сервер;
- клиент-сервер (многозвенная);
- сервис-ориентированная;
- сервис-ориентированная (шина данных).

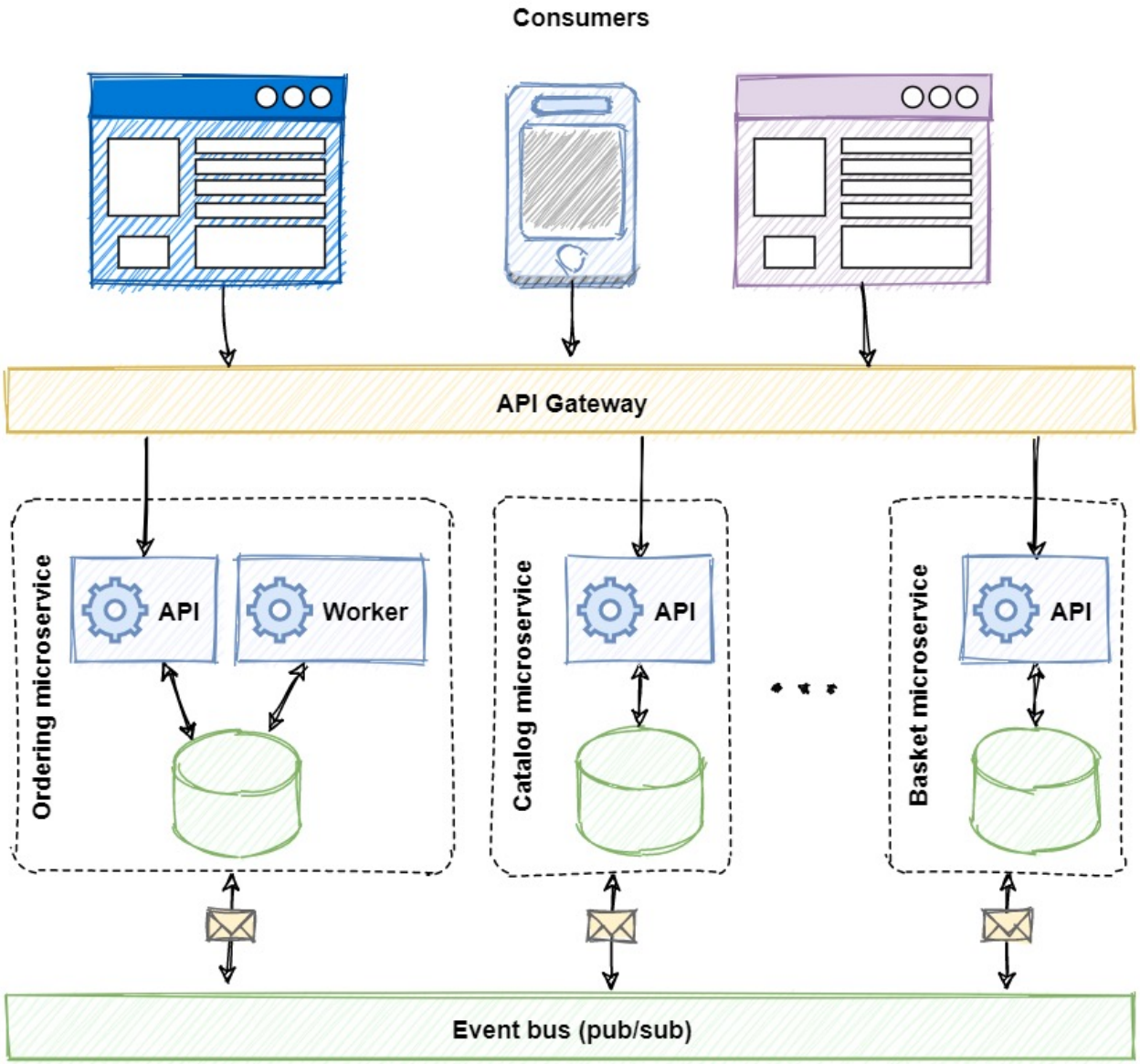
Клиент-сервер (3-х звенная)



Сервис-ориентированная архитектура (шина) – 1



Сервис-ориентированная архитектура (шина) – 2



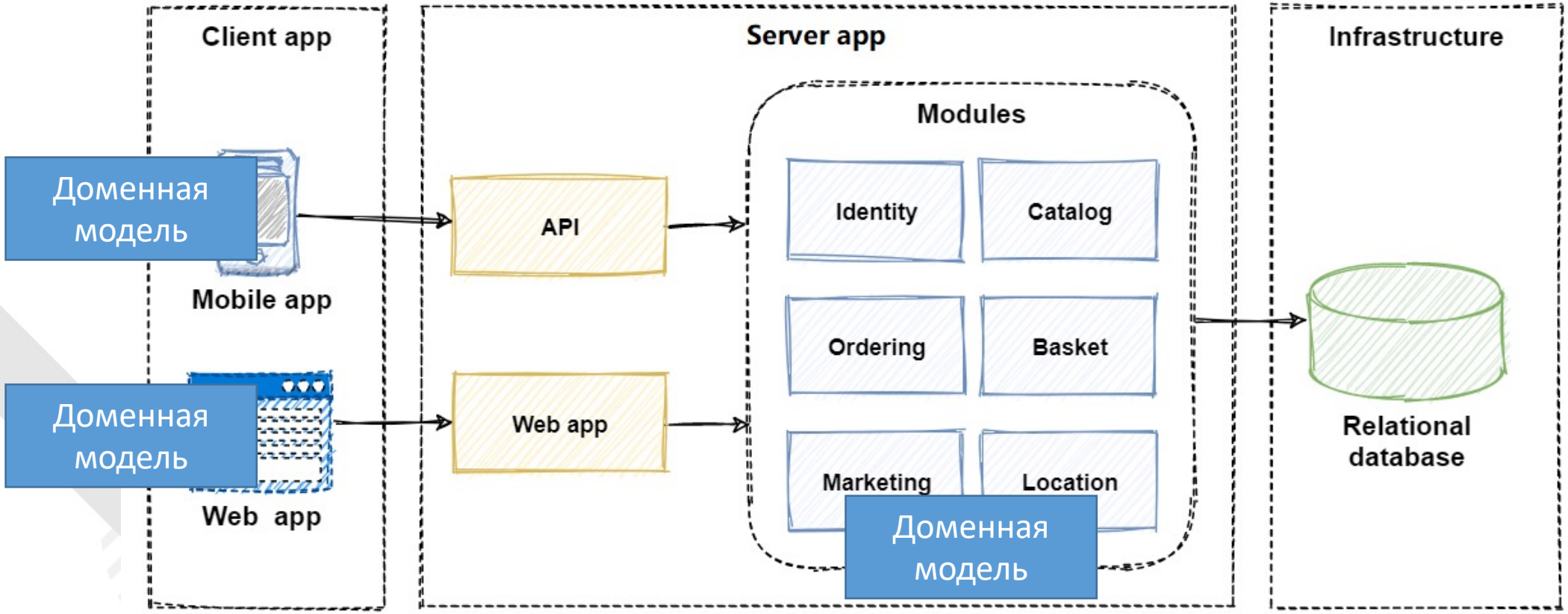
Предметно-ориентированное проектирование

Предметно-ориентированное проектирование (domain-driven design, DDD) – подход к разработке программного обеспечения, в котором упор делается на реализацию бизнес-логики, а не на технологические особенности системы.

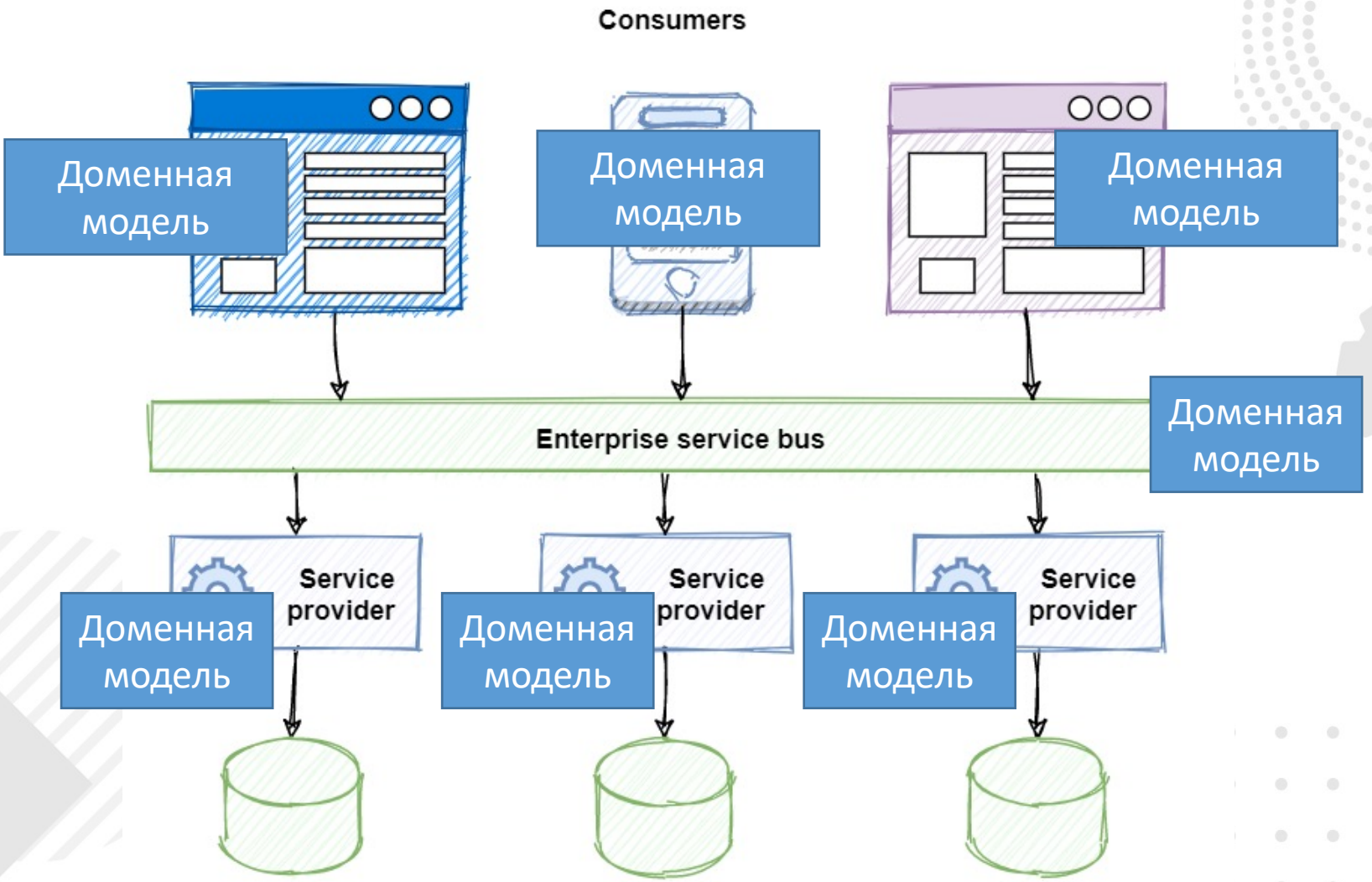
При использовании DDD необходимо организовать общение между разработчиками и представителями заказчика, в некоторых случаях может быть привлечен аналитик.

Центральное место в программной системе занимает модель данных и бизнес-логика.

Доменная модель – Клиент-сервер



Доменная модель – СОА



Доменная модель

Data access object (DAO) – абстрактный интерфейс к какому-либо типу базы данных или механизму хранения.

DAO описывает возможности некоторого хранилища без привязки к его реализации.

DAO абстрагирует работу с конкретным типом хранилища, давая возможность легкой замены
реализации хранилища:

- смена СУБД;
- смена типа хранилища.

Шаблон фасад (Facade) – структурный шаблон проектирования, позволяющий скрыть сложность системы путем сведения всех возможных внешних вызовов к одному объекту.

Фасад скрывает реализацию некоторой подсистемы от других компонентов программной системы.

Доменная модель в распределенных приложениях

Проблемы применения предметно-ориентированного проектирования в распределенных системах:

- Взаимодействие с удаленными источниками данных.
- Преобразование данных из удаленных источников в доменную модель.
- Сочетание различных источников данных.

Data Transfer Object (DTO) – шаблон проектирования для передачи данных между подсистемами приложения.

DTO НЕ должен содержать какого-либо поведения.

REST

REST (Representational State Transfer, передача репрезентативного состояния) – архитектурный стиль взаимодействия компонентов распределенного приложения в сети.

REST – это набор правил организации взаимодействия между программными системами.

В определенных случаях использование REST приводит к повышению производительности и упрощению архитектуры.

При использовании REST взаимодействие между системами строится на основе HTTP-запросов (REST-запросов): GET, POST, DELETE, PATCH, PUT. Необходимые данные передаются в качестве параметров запроса.

Для REST API не существует «официального» стандарта, так как REST является архитектурным стилем.

REST API обычно базируется на следующих стандартах: HTTP, URL, JSON (реже XML).

Преимущества REST

Применение REST API позволяет добиться масштабируемости приложения за счет:

1. Простого унифицированного интерфейса.
2. Открытости компонентов к возможным изменениям для удовлетворения изменяющихся потребностей.
3. Прозрачности связей между компонентами системы для сервисных служб.
4. Переносимости компонентов системы путем перемещения программного кода вместе с данными.
5. Надёжности, выражающейся в устойчивости к отказам на уровне системы при наличии отказов отдельных компонентов, соединений или данных.

Шесть ограничений REST

Архитектура REST описывается шестью ограничениями:

1. Единый интерфейс (Uniform Interface)
2. Отсутствие состояний (Stateless)
3. Кеширование ответа (Cacheable)
4. Клиент-сервер (Client-Server)
5. Многоуровневая система (Layered System)
6. "Код по требованию" (Code on Demand - опционально)

HTTP методы в REST

REST интерфейс обычно основан на следующих HTTP методах:

1. GET – получение данных о сущности/сущностях.
2. POST – создание новой сущности.
3. PUT/PATCH – модификация сущности.
4. DELETE – удаление сущности.

C – POST
R – GET
U – PUT/PATCH
D – DELETE

Именование ресурсов в REST

REST API разрабатывается для потребителей. Названия и структура URI должна передавать смысл доменной модели.

Примеры:
POST http://www.example.com/customers
GET http://www.example.com/customers/33245
GET|PUT|DELETE http://www.example.com/products/66432
GET http://www.example.com/customers/33245/orders
GET http://www.example.com/customers/33245/orders/8769/lineitems/1

Ошибки:
GET http://api.example.com/services?op=update_customer&id=12345&format=json
GET http://api.example.com/update_customer/12345
GET http://api.example.com/customers/12345/update
PUT http://api.example.com/customers/12345/update

Версии API в REST

Версионирование REST API позволяет сохранять обратную совместимость при изменении API для поддержки старых клиентов.

http://www.example.com/1.0/customer
http://www.example.com/1.0/customer/33245

http://www.example.com/2.0/customers
http://www.example.com/2.0/customers/33245

JSON

JSON (JavaScript Object Notation) – формат представления данных. Данные представлены в виде объектов на языке JavaScript.

Сериализация — процесс перевода структуры данных в последовательность байтов.

Десериализация – создание структуры данных из битовой последовательности.



OpenAPI Specification

The OpenAPI Specification (Swagger Specification) — универсальный интерфейс для пользователей (клиентов) по взаимодействию с сервисами (серверами).

Если спроектирована спецификация для некоторого сервиса, то на её основании можно генерировать исходный код для библиотек клиентских приложений, текстовую документацию для пользователей, варианты тестирования и др.

OpenAPI представляет собой формализованную спецификацию и полноценный фреймворк для описания, создания, использования и визуализации веб-сервисов REST.

Спецификация не зависит от языка программирования, и может быть использована вне протокола HTTP. OpenAPI одновременно применяется для клиента, сервера и соответствующей документации интерфейса, созданного согласно REST.

Spring Boot – Доменная модель

Фреймворк Spring Boot предоставляет набор средств для формирования доменной модели. Доменная модель включает в себя следующие компоненты:

1. Сущности предметной области (JPA, ORM, Hibernate).
2. Слой бизнес-логики (@Service).
3. DAO (@Repository).
4. Валидация данных (Hibernate Validator).

Spring Boot – REST

Фреймворк Spring Boot позволяет формировать REST API с помощью следующих компонентов:

1. REST-контроллер (@RestController).
2. Spring Doc (OpenApi).
3. Сериализация и десериализация (Jackson: JSON, XML).
4. Валидация данных (jakarta.validation).

Пагинация (Paging) – 1

Пагинация используется для получения и отображения страниц с данными из локального хранилища или сети интернет при наличии крупного массива данных.

Пагинация позволяет эффективно использовать ресурсы системы и пропускную способность канала связи.

Пагинация (Paging) – 2

Фреймворк Spring Boot позволяет реализовать пагинацию на уровне работы с хранилищем через следующие интерфейсы-репозитории:

1. PagingAndSortingRepository.
2. JpaRepository.

Данные интерфейсы позволяют использовать специальную функцию:

```
Page<T> findAll(Pageable pageable)
```

Также можно добавлять поддержку пагинации в собственные функции репозитория:

```
public interface SomeRepository extends
    CrudRepository<SomeEntity, Long>,
    PagingAndSortingRepository<SomeEntity, Long> {

    Page<SomeEntity> findByUserId(long userId, Pageable pageable);
}
```

Пагинация (Paging) – 3

Пример вызова функции из репозитория с поддержкой пагинации:

```
Page<SomeEntity> result = repository.findByUserId(userId, PageRequest.of(page, size));
```

page – номер страницы;
size – размер страницы.

Все множество данных делится на n страниц размера size.

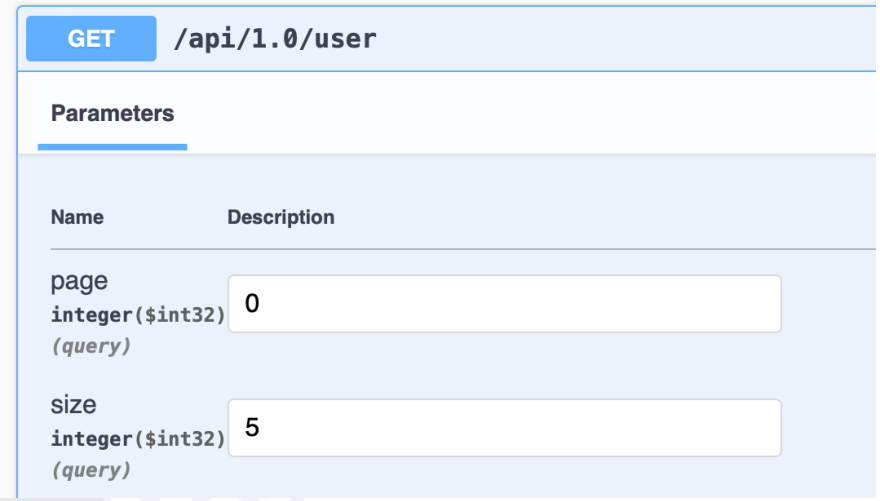
Пагинация реализуется на уровне СУБД через операторы limit и offset.

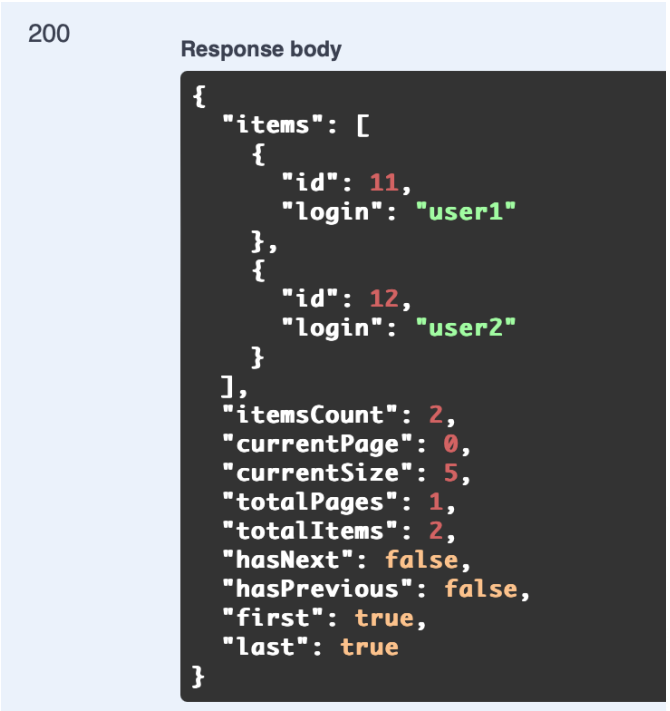
Результат представляет собой объект класса Page, из которого можно получить:

- коллекцию элементов;
- количество элементов;
- номер текущей страницы;
- размер страницы;
- количество страниц;
- количество элементов в странице;
- признак первой и последней страницы;
- признак наличия следующе и предыдущей страницы и др.

Пагинация (Paging) – 4

```
@GetMapping
public PageDto<UserDto> getAll(
      @RequestParam(name = "page", defaultValue = "0") int page,
      @RequestParam(name = "size", defaultValue = ”5") int size) {
   return PageDtoMapper.toDto(userService.getAll(page, size), this::toDto);
}
```





Обработка ошибок – 1

Фреймворк Spring Boot позволяет перехватывать и выводить сообщения об ошибках при взаимодействии с сервисом.

Реализация по умолчанию выводит сообщение об ошибке в следующем формате:

```
{
   "timestamp": "2019-01-17T16:12:45.977+0000",
   "status": 500,
   "error": "Internal Server Error",
   "message": "Error processing the request!",
   "path": "/my-endpoint-with-exceptions"
}
```

Обработка ошибок – 2

Также можно переопределить стандартное поведение.

```
@RestControllerAdvice
public class AdviceController {

    @ExceptionHandler(NotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ResponseEntity<String> handleNotFoundException(Throwable throwable) {
        return new ResponseEntity<>(throwable.getMessage(), HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(Exception.class)
    @ResponseStatus(HttpStatus.INTERNAL_SERVER_ERROR)
    public ResponseEntity<String> handleAnyException(Throwable throwable) {
        return new ResponseEntity<>(throwable.getMessage(),
                                    HttpStatus.INTERNAL_SERVER_ERROR);
    }

}
```

Профили

Профили позволяют конфигурировать приложения для особенностей выполнения в рамках конкретной среды:

- основной сервер (production);
- сервер разработки (development);
- сервер тестирования (test) и т. д.

Также профили позволяют разделить конфигурацию приложения на отдельные части и комбинировать части конфигурации для повышения модульности проекта:

- работа с разными СУБД;
- работа с разными механизмами аутентификации и авторизации;
- работа с разными шаблонизаторами и т. д.

Профили в Gradle – 1

```
ext {
    springProfiles = []
    if (project.hasProperty('prod')) {
        springProfiles.add('prod')
    } else {
        springProfiles.add('dev')
    }
}

dependencies {
    if (project.hasProperty('prod')) {
        implementation 'org.postgresql:postgresql:42.7.3'
    } else {
                implementation 'com.h2database:h2:2.2.224'
    }
    ...
}
```

Профили в Gradle – 2

```
bootRun {
    def currentProfiles = springProfiles.join(',')
    println('Current profiles are: ' + currentProfiles)
    def currentArgs = ['--spring.profiles.active=' + currentProfiles]
    if (project.hasProperty('args')) {
        currentArgs.addAll(project.args.split(','))
    }
    args currentArgs
}
```

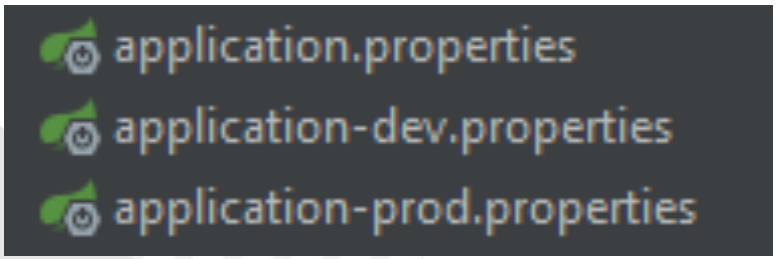
Пример запуска приложения с нужным профилем и аргументами:

```
gradlew -Pprod -Pargs='--populate'
```

Профили в Spring

application.properties

```
# Logger settings
logging.level.com.example.demo=DEBUG
# Set default orofile
spring.profiles.default=dev
# Common JPA Settings
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.open-in-view=false
```



application-dev.properties

```
# Dev H2 JPA Settings
spring.datasource.url=jdbc:h2:file:./data
spring.datasource.username=sa
spring.datasource.password=password
spring.datasource.driver-class-
name=org.h2.Driver
# H2 console
spring.h2.console.enabled=true
```

application-prod.properties

```
# Logger settings
logging.level.com.example.demo=INFO
# Prod PostgreSQL JPA Settings
spring.datasource.url=jdbc:postgresql://127.0.0.1
:5432/demo-app
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.datasource.driver-class-
name=org.postgresql.Driver
```

Миграции

Схема базы данных — структура БД, описанная на формальном языке, поддерживаемом СУБД. Включает описание содержания и ограничений целостности.

Таблица — связанные данные, которые в структурированном виде хранятся в БД. Состоит из столбцов и строк.

Миграция данных:

- Разовое массовое перемещение данных из систем-источников в систему-приемник. Эксплуатация данных в системах-источниках при этом прекращается.
- **Процесс преобразования БД, при котором меняется схема БД от одной версии до другой. Миграция подразумевает как прямое изменение БД (апгрейд, db_upgrade), так и обратное изменение (даунгрейд, db_downgrade).**

Необходимость миграции

Синхронизация кода проекта и БД при создании приложения несколькими разработчиками (наподобие системы контроля версий).

Версионная миграция данных — обновление структуры БД до новой версии, чтобы версии приложения и БД соответствовали друг другу для правильной работы.

Версионная миграция данных – 1

Версионная миграция данных нужна во время разработки, чтобы не столкнуться с проблемой рассинхронизации версий БД и приложения при поставке новой версии приложения.

Обновление версии приложения требует корректной версионной миграции БД.

Корректность заключается в том, чтобы в процессе версионной миграции все SQL-запросы выполнялись лишь единожды и строго в определенной последовательности, так как одни изменения могут зависеть от других.

Версионная миграция данных – 2

Основные принципы версионной миграции данных:

- любая версия БД должна быть обновлена до любой версии;
- набор SQL-запросов, которые позволяют реализовать миграцию между любыми двумя версиями, должны быть получены максимально быстро и просто;
- должна быть возможность создать БД с нуля со структурой самой последней версии;
- слияние разных веток после работы над ними ручное редактирование файлов БД должно быть сведено к минимуму;
- откатить БД на более раннюю версию должно быть так же просто, как и обновить на более новую.

Миграция данных в Java

Liquibase – это проект с открытым исходным кодом, который помогает миллионам разработчиков быстро управлять изменениями схемы базы данных.

implementation 'org.liquibase:liquibase-core:4.26.0'

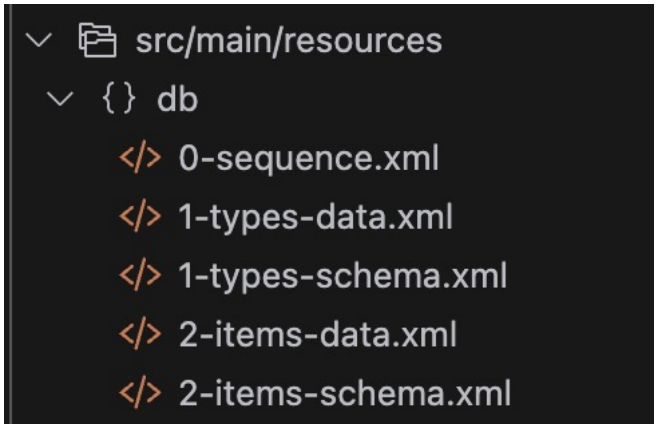
Основные функции:

- Гибкие возможности изменения схемы.
- Автоматическая генерация SQL-скриптов.
- Повторяемые миграции.
- Интеграция со сторонними средствами.
- Механизм откатов.
- Контекстно-зависимая логика.
- Поддержка широкого спектра СУБД.

Пример реализации миграции
Настройки приложения

application.properties

```
spring.liquibase.enabled=true
spring.liquibase.drop-first=false
spring.liquibase.change-log=classpath:db/changelog.xml
```



Основной файл миграции

changelog.xml

```
<databaseChangeLog xmlns="http://www.liquibase.org/xml/ns/dbchangelog"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://www.liquibase.org/xml/ns/dbchangelog
        http://www.liquibase.org/xml/ns/dbchangelog/dbchangelog-latest.xsd">

    <include file="db/0-sequence.xml" />
    <include file="db/1-types-schema.xml" />
    <include file="db/1-types-data.xml" />
    <include file="db/2-items-schema.xml" />
    <include file="db/2-items-data.xml" />

</databaseChangeLog>
```

Создание таблицы

items-schema.xml

```
<changeSet author="user" id="items-schema">
    <createTable tableName="items">
        <column name="id" type="bigint">
            <constraints nullable="false" />
        </column>
        <column name="type_id" type="bigint">
            <constraints nullable="false" />
        </column>
        <column name="price" type="double">
            <constraints nullable="false" />
        </column>
        <column name="count" type="integer">
            <constraints nullable="false" />
        </column>
    </createTable>
    <addPrimaryKey columnNames="id" constraintName="pk_items" tableName="items" />
    <addForeignKeyConstraint constraintName="fk_item_type_id"
        baseTableName="items" baseColumnNames="type_id"
        referencedTableName="types" referencedColumnNames="id" />
</changeSet>
```

Заполнение таблицы

items-data.xml

```
<changeSet author="user" id="items-data">
    <insert tableName="items">
        <column name="id" value="4" />
        <column name="type_id" value="1" />
        <column name="price" value="49999.00" />
        <column name="count" value="20" />
    </insert>
    <insert tableName="items">
        <column name="id" value="5" />
        <column name="type_id" value="1" />
        <column name="price" value="129999.00" />
        <column name="count" value="3" />
    </insert>
    ...
    <insert tableName="items">
        <column name="id" value="10" />
        <column name="type_id" value="3" />
        <column name="price" value="67800.00" />
        <column name="count" value="3" />
    </insert>
</changeSet>
```

Спасибо за внимание!

Литература:
1. Руководство по REST API. URL: https://restapitutorial.ru/
2. Paging and sorting with Spring Data JPA. URL: https://www.baeldung.com/spring-data-jpa-pagination-sorting
3. Error Handling for REST with Spring. URL: https://www.baeldung.com/exception-handling-for-rest-with-spring\
4. Использование Liquibase без головной боли. 10 советов из опыта реальной разработки. URL: https://habr.com/ru/articles/179425/



2022